

AIM- To model a limited shared resource using a FreeRTOS counting semaphore and study access control when multiple tasks request the resource simultaneously.

APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, Ultrasonic Sensor, STM32 CUBE IDE, STM32 CubeMX.

THEORY

In Real-Time Operating Systems (RTOS), managing access to shared hardware or software resources is a fundamental challenge. When multiple tasks—each representing a distinct thread of execution—attempt to utilize a finite resource simultaneously, the system must employ a synchronization mechanism to ensure data integrity, prevent hardware conflicts, and maintain predictable timing. For a single resource, a binary semaphore or mutex is sufficient; however, when a system possesses multiple identical resources, a **Counting Semaphore** is the required primitive.

The Nature of Counting Semaphores

A counting semaphore can be envisioned as a pool of identical tokens. It is initialized with two values: a **Maximum Count** (representing the total available units of a resource) and an **Initial Count**. In the context of this experiment, such as a robotic platform with two identical actuator arms, the semaphore is initialized with a count of **2**.

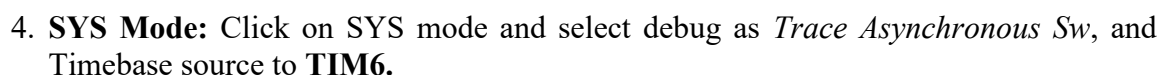
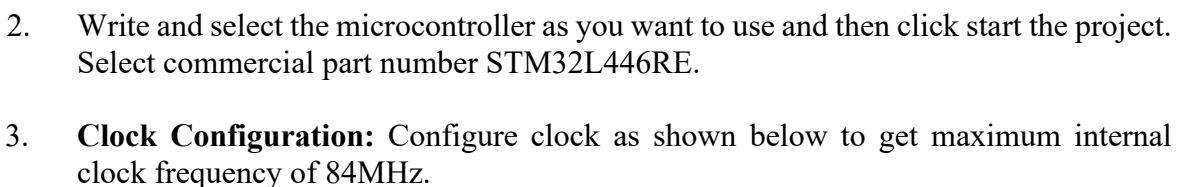
When a task requires a resource, it performs a **'Take'** (or *Acquire*) operation. If the semaphore count is greater than zero, the count is decremented, and the task proceeds. If the count is zero, indicating all resources are occupied, the task enters the **Blocked** state, relinquishing CPU time until a unit becomes available or a timeout occurs. Conversely, when a task finishes with the resource, it performs a **'Give'** (or *Release*) operation, incrementing the count and potentially waking a blocked high-priority task.

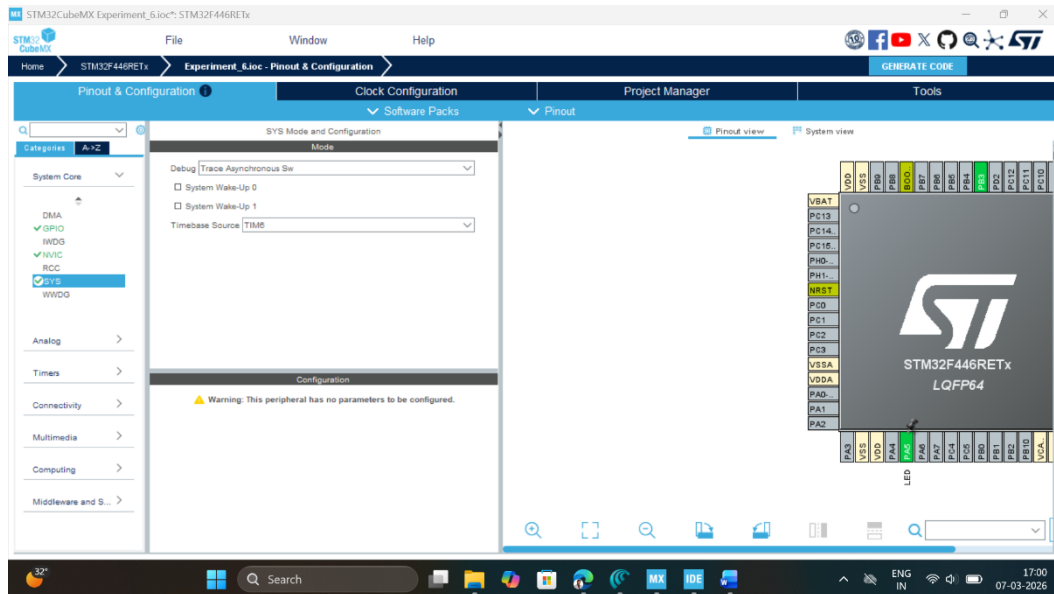
Resource Contention and Interleaving

Unlike a Mutex, which provides exclusive "Mutual Exclusion" (allowing only one owner), a counting semaphore with a count >1 allows for **parallelism**. While this increases throughput—for example, allowing Task A and Task B to print to a trace buffer simultaneously—it introduces the risk of **interleaving**. Since the underlying communication hardware (like a single UART or ITM port) is often a singleton, multiple tasks writing to it at once will result in garbled data or "racing" characters. This demonstrates that while the semaphore manages the *quantity* of access, it does not inherently protect the *atomicity* of the data stream.

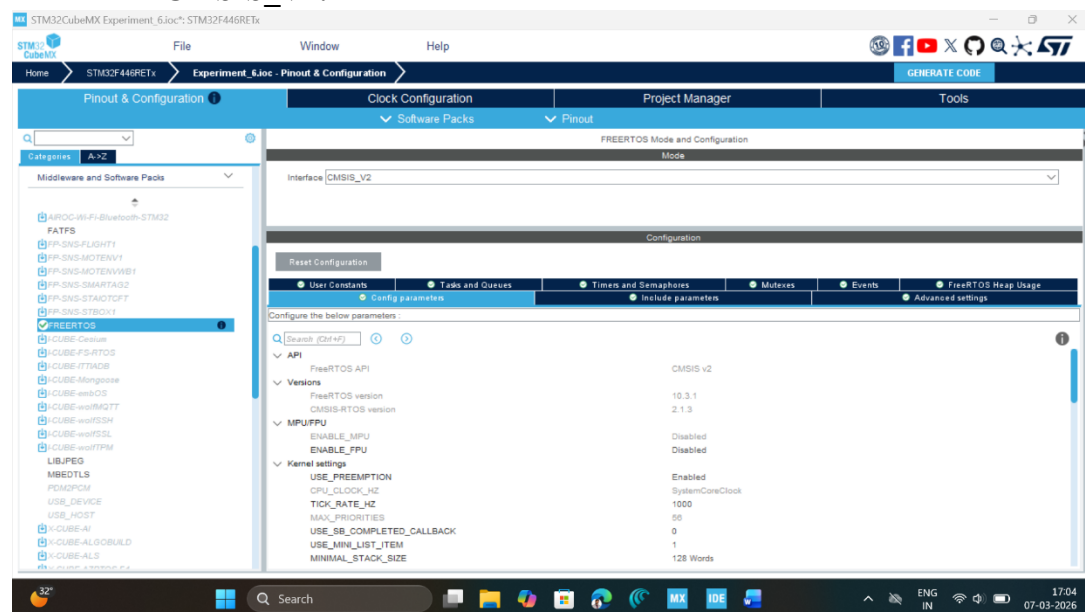
PROCEDURE

1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.





5. **FreeRTOS Configuration:** Click on Middleware and Software Pack, then select FreeRTOS software pack, which is already available with STMCube Mx. Select interface as CMSIS_V2.



6. Click on the Tasks & Queues Tab of configuration settings and rename the default task as 'TaskA'. Rename the entry function as 'func_TaskA()'. Add two more tasks named 'TaskB' and 'TaskC', and rename their entry functions as 'func_TaskB()' and 'func_TaskC()' respectively.

Configuration

[Reset Configuration](#)

☒ Mutexes	☒ Events	☒ FreeRTOS Heap Usage
☒ User Constants	☒ Tasks and Queues	☒ Timers and Semaphores
☒ Config parameters	☒ Include parameters	☒ Advanced settings

Tasks

Task Na...	Priority	... Entry Fu...	Code Ge...	Parameter	Allocation	Buffer N...	Control ...
TaskA	osPriorityNormal	... func_Ta...	Default	NULL	Dynamic	NULL	NULL
TaskB	osPriorityNormal	... func_Ta...	Default	NULL	Dynamic	NULL	NULL
TaskC	osPriorityNormal	... func_Ta...	Default	NULL	Dynamic	NULL	NULL

[Add](#)
[Delete](#)

7. Click on the Timers and Semaphores Tab of configuration settings to 'Add' a Counting Semaphore to the application by clicking the Add button as shown below:

Configuration

[Reset Configuration](#)

☒ Mutexes	☒ Events	☒ FreeRTOS Heap Usage
☒ User Constants	☒ Tasks and Queues	☒ Timers and Semaphores
☒ Config parameters	☒ Include parameters	☒ Advanced settings

Timers

Timer Name	Callback	Type	Code Gener...	Parameter	Allocation	Control Blo...
------------	----------	------	---------------	-----------	------------	----------------

[Add](#)
[Delete](#)

Binary Semaphores

Semaphore Name	Allocation	Control Block Name	Initial State
----------------	------------	--------------------	---------------

[Add](#)
[Delete](#)

Counting Semaphores

Semaphore Name	Max Count	Allocation	Control Block Name	Initial Count
myCountingSem01	2	Dynamic	NULL	2

[Add](#)
[Delete](#)

8. Select Advanced Settings tab, and enable the USE_NEWLIB_REENTRANT setting

Configuration

[Reset Configuration](#)

☒ Timers and Semaphores	☒ Mutexes	☒ Events	☒ FreeRTOS Heap Usage
☒ Config parameters	☒ Include parameters	☒ Advanced settings	☒ User Constants
☒ Tasks and Queues			

Configure the below parameters :

Search (Ctrl+F) [Info](#)

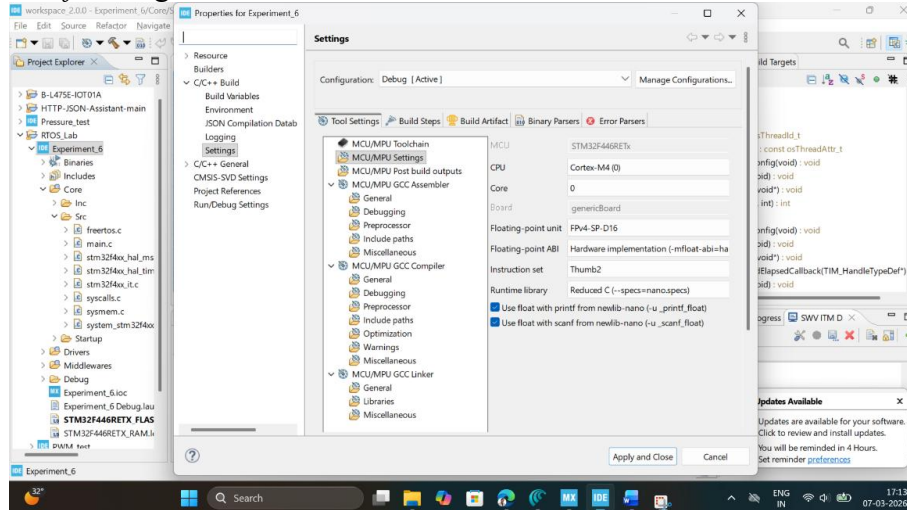
- ☒ Newlib settings (see parameter description first)

USE_NEWLIB_REENTRANT	Enabled
----------------------	---------
- ☒ Project settings (see parameter description first)

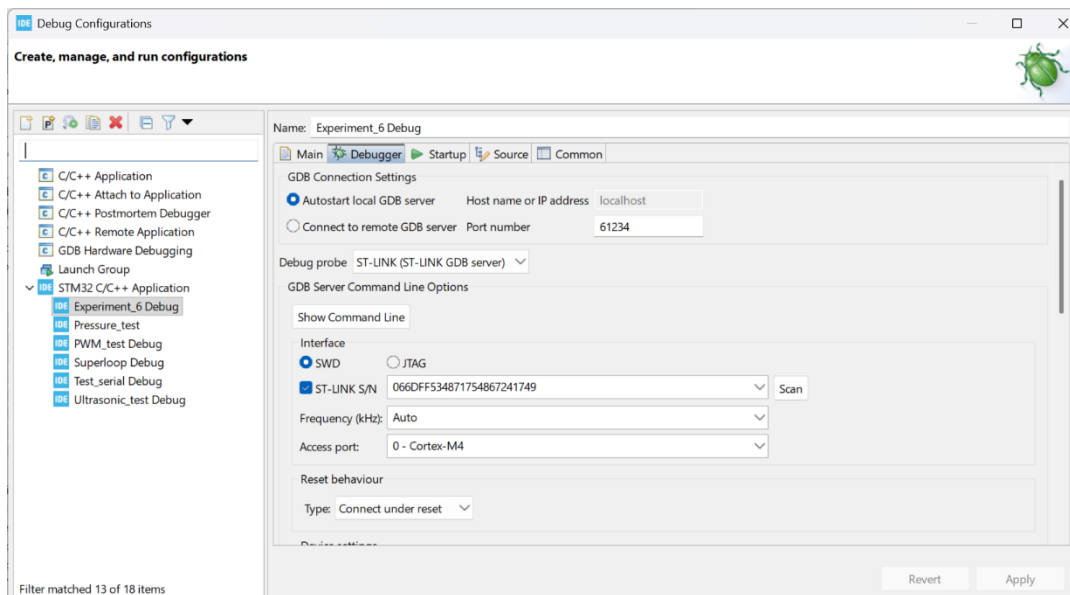
Use FW pack heap file	Enabled
-----------------------	---------

9. Write Project name and select IDE.

10. Click on generate code.
11. Click on the open project.
12. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
13. Open the main source code on Cube IDE by clicking on **main.c**
14. Click on project properties, go to C/C++ Build settings check box for *printf* and *scanf* settings.



15. Now click on Debug icon and configure debugger for SWV ITM Trace
16. Click on Debugger configuration, select the debugger console, and click on ST-LINK S/N and then scan, as we attach a STM32F446RE development board, it will automatically pick up ST-LINK id.



17. Next, select the Serial Wire Viewer (SWV) and check the enable button and make sure to pass on the right core clock frequency which is set-up at point 4 here i.e. 84MHz. Then click on the Apply button and close this window.

18. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling. */* Private includes -----*

```
-----*/
/* USER CODE BEGIN Includes */
#include<stdio.h>
/* USER CODE END Includes */
```

Write the function to enable SWV ITM Console to trace the output of tasks

```
/* USER CODE BEGIN 0 */
int _write(int file, char *ptr, int len) {
for (int i = 0; i < len; i++) {
ITM_SendChar(*ptr++);
}
return len;
}
/* USER CODE END 0 */
```

Edit the task functions as given below:

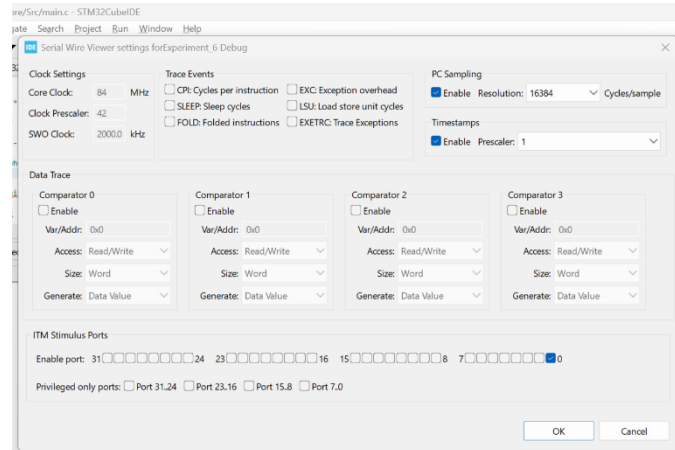
```
void func_TaskA(void *argument)
{
/* USER CODE BEGIN 5 */
    char ch = 'A';
/* Infinite loop */
for(;;)
{
    //Access to trace output is limited by a semaphore
    osSemaphoreAcquire(myCountingSem01Handle, osWaitForever);
    //printf("1AAAAAAAAAA\n");
    printf("1");
    for(int i = 0; i < 10; i++){
        printf("%c", ch);
        HAL_Delay(50);
    }
    osSemaphoreRelease(myCountingSem01Handle);    //Give Semaphore
    osDelay(5);
}
/* USER CODE END 5 */
}
/* USER CODE BEGIN Header_func_TaskB */
/**
 * @brief Function implementing the TaskB thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_func_TaskB */
void func_TaskB(void *argument)
{
/* USER CODE BEGIN func_TaskB */
    char ch = 'B';
/* Infinite loop */
```

```

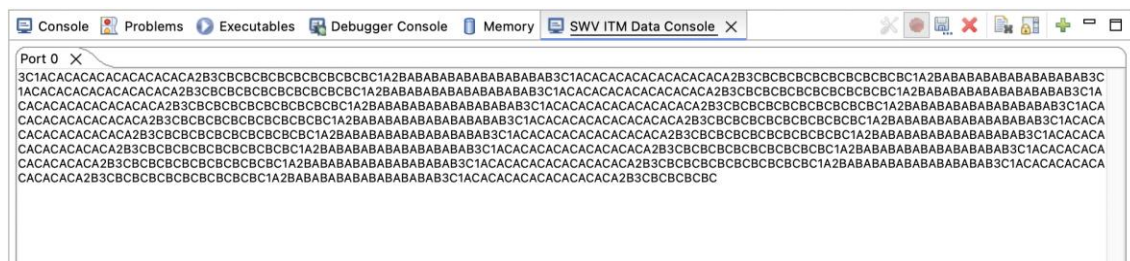
for(;;)
{
    //Access to trace output is limited by a semaphore
    osSemaphoreAcquire(myCountingSem01Handle, osWaitForever);
    printf("2");
    for(int i = 0;i < 10;i++){
        printf("%c",ch);
        HAL_Delay(50);
    }
    osSemaphoreRelease(myCountingSem01Handle); //Give Semaphore
    osDelay(5);
}
/* USER CODE END func_TaskB */
}
/* USER CODE BEGIN Header_func_TaskC */
/**
 * @brief Function implementing the TaskC thread.
 * @param argument: Not used
 * @retval None
 */
/* USER CODE END Header_func_TaskC */
void func_TaskC(void *argument)
{
    /* USER CODE BEGIN func_TaskC */
    char ch = 'C';
    /* Infinite loop */
    for(;;)
    {
        //Access to trace output is limited by a semaphore
        osSemaphoreAcquire(myCountingSem01Handle, osWaitForever);
        printf("3");
        for(int i = 0;i < 10;i++){
            printf("%c",ch);
            HAL_Delay(50);
        }
        osSemaphoreRelease(myCountingSem01Handle); //Give Semaphore
        osDelay(5);
    }
    /* USER CODE END func_TaskC */
}

```

19. Now, click on the **Window** tab, select **Show view** → **other** and select SWV ITM Data Console.
20. Now click on the Debug button, start debugging this main.c file and click on switch to change the perspective to debugger.
21. Click on SWV ITM Data Console setting, enable the PC sampling and select port 0.



22. Click on Start Trace (Red Dot) on SWV ITM Data Console tab and then click on resume button to start executing the main program.



OBSERVATION:

Observation Number	Query	Response
1	How many tasks are able to use the ITM trace resource concurrently?	Since the counting semaphore is initialized with a count value of 2 , at most two tasks can access the ITM trace resource simultaneously. The third task remains blocked until one semaphore token is released.
2	Comment on the interleaving of tasks by observing the trace output.	The SWV ITM trace output shows mixed and overlapping characters from different tasks such as AAAA, BBBB, and CCCC appearing together. This interleaving occurs because two tasks are allowed to access the same ITM output channel at same time. As a result, characters from multiple tasks become mixed, producing garbled output stream.
3	Identify the type of scheduling that lead to interleaving of tasks.	The interleaving is caused by preemptive multitasking with time-sliced scheduling used in FreeRTOS. The scheduler frequently switches between read tasks of equal priority, causing their outputs to overlap while sharing the ITM resource.
4	Tinker with the task code and present any different outcome than that observed in the beginning.	When the counting semaphore value was changed from 2 to 1 , the output became more organized. Only one task accessed the ITM resource at a time, resulting in complete blocks such as AAAAAAAAAA, BBBBBBBBBB, and CCCCCCCC without mixing. This behavior became similar to a mutex or binary semaphore, ensuring mutual exclusion.

RESULT

The experiment successfully demonstrated the implementation of a **FreeRTOS Counting Semaphore** for controlling access to a limited shared resource. Three tasks attempted to access the same ITM trace output simultaneously, while the semaphore restricted concurrent access according to its count value.

With the semaphore count set to **2**, two tasks were able to access the resource at the same time, leading to interleaved and mixed outputs in the SWV ITM Data Console. This verified that counting semaphores manage the number of concurrent users but do not guarantee atomic access to a communication channel.

The experiment also highlighted the effect of RTOS scheduling and context switching in multitasking systems. By modifying the semaphore count to **1**, orderly and non-interleaved output was achieved, proving the importance of proper synchronization mechanisms in embedded RTOS applications.

REFLECTION SUMMARY

This experiment provided a strong practical understanding of synchronization and resource management in RTOS-based embedded systems. I learned how multiple tasks running concurrently can create conflicts when accessing a shared hardware resource such as the ITM trace console. The use of a FreeRTOS counting semaphore helped in controlling the number of tasks allowed to access the resource simultaneously.

Through the SWV ITM trace output, I observed how task scheduling and context switching lead to interleaving of data when synchronization is not fully exclusive. This helped me understand the real difference between a counting semaphore and a mutex. A counting semaphore improves resource utilization and throughput by allowing limited parallel access, whereas a mutex ensures strict mutual exclusion for orderly communication.

I also learned how FreeRTOS task scheduling works internally, especially preemptive scheduling and time slicing among tasks of equal priority. The experiment improved my understanding of multitasking behavior, semaphore operations (Acquire and Release), and debugging using SWV ITM Trace in STM32CubeIDE.

Overall, this experiment strengthened my concepts of RTOS synchronization, task management, and embedded system design, which are essential for developing reliable real-time applications in modern microcontroller-based systems.